

DTS 301 PROJECT

Smart-Grid Energy Consumption Forecasting

Group members		Contribution
Gabriel Theophilus	Run/cmp/23/15347	Built the actual model and presented
Ali peace Ojonemile	Run/cmp/23/15203	Presented and worked on the Project development workflow, Dataset description and system architecture of the documentation.
Ofili Daniel	Run/cmp/23/15290	Worked on Model Evaluation and Results in Energy Consumption Forecasting for Smart Grids in documentation
Negedu Israel	Run/cmp/23/15283	Built the demo model
Adegbite John Oluwadamilola	Run/cmp/23/15170	Worked on the installation and usage part of the documentation
Ibiyinka Adebola	Run/cmp/23/15256	Worked on the objective part of the documentation
Adelabu Adebowale	Run/cmp/23/15172	Worked on the conclusion part of the documentation
Adeola Toluwase	Run/cmp/23/15174	Worked on the PowerPoint presentation slide
Eigbiluese Jude	Run/cmp/23/15234	
Aderibigbe Inioluwa	Run/cmp/23/15175	

Contents

1. Project Description & Objectives.....	3
1.1 Introduction.....	3
1.2 Problem Statement.....	3
1.3 Objective of energy consumption forecasting in smart grids.....	3
2. Project Development Workflow.....	4
2.1 The repo's structure:.....	4
2.2 Technologies.....	4
3 Dataset Description.....	4
3.1 Dataset Source.....	4
3.2 Data Format.....	5
3.3 Data Characteristics.....	5
3.4 How the Dataset is used in this project.....	6
4. System Architecture.....	6
4.1 System Workflow.....	6
4.2 Major Components of the Architecture.....	7
5 Model Evaluation and Results in Energy Consumption Forecasting for Smart Grids.....	9
5.1 Training Data Preparation.....	9
5.2 Evaluation Metrics.....	9
5.3 Training Results.....	10
5.4 Training vs Validation Analysis.....	10
5.5 Model Stability and Generalization.....	10
6. Installation and Usage.....	10
6.1 Environment Setup and System Execution.....	10
6.2 Tools and Framework Usage.....	11

Smart-Grid Energy Consumption Forecasting — Project Documentation

1. Project Description & Objectives

1.1 Introduction

Energy consumption forecasting is an essential task in modern smart-grid systems because it helps grid operators plan energy distribution, reduce overload, and improve efficiency. With the increasing adoption of smart meters and IoT sensors, large volumes of real-time energy data are now generated every second. To make sense of this data and accurately predict future consumption, advanced data processing and machine learning techniques are required.

The goal of this project is to forecast real-time energy consumption in a smart grid. This project implements a machine learning system for forecasting energy consumption using historical time-series data. The system employs deep learning (LSTM) and traditional time-series models to predict future energy usage based on historical patterns.

This project focuses on building a smart-grid forecasting system using **Apache Spark Streaming** for real-time data ingestion and **machine-learning models** such as **LSTM neural networks** and **ARIMA time-series models**. The system also incorporates external factors like weather and calendar information to improve prediction accuracy. By combining these methods in an ensemble, the project provides a more reliable forecast of future energy demand.

1.2 Problem Statement

Accurate and timely energy consumption forecasting is crucial for optimising smart grid operations, preventing grid instability, and enhancing resource allocation. The challenge lies in processing high-volume, real-time sensor data and building sophisticated models capable of capturing both the inherent time-series dependencies and the influence of complex external variables.

1.3 Objective of energy consumption forecasting in smart grids

The primary objective of energy consumption forecasting in smart grids is to predict future electricity demand, enabling utility companies to manage power generation and distribution more efficiently. The specific objectives include:

1. Prevent power outages and blackouts.
2. Optimise energy distribution across the smart grid to balance supply and demand

3. Reduce wastage of electricity by supplying only what's needed.
4. Support real-time decision-making for effective energy management.
5. Improve operational efficiency of power plants and grids.
6. Incorporate weather and seasonal factors into demand prediction.
7. Provide accurate forecasts for households, industries, and commercial areas.
8. Support smart meter integration for continuous monitoring and data-driven insights.
9. Allow electricity companies to handle unexpected surges in demand
10. Help utility companies plan for peak load hours

2. Project Development Workflow

2.1 The repo's structure:

The repository structure is as follows

energy forecasting/

- ├── data/ # sourced data from Kaggle
- ├── models/ # LSTM, ARIMA, and ensemble model implementations
- ├── training/ # model training pipelines
- ├── streaming/ # Spark Streaming processors (real-time ingestion & processing)
- ├── monitoring/ # Web dashboard and monitoring tools
- ├── config/ # configuration settings
- └── main.py # Main orchestrator

2.2 Technologies

CATEGORY	TECHNOLOGY
Distributed Computing	Apache Spark & PySpark
Deep Learning	TensorFlow/Keras (LSTM)
Time-Series Analysis	Statsmodel (ARIMA)
Implementation / Usage	React JS and Python Flask
Data Handling	Pandas, Numpy, Scikit-learn

3 Dataset Description

3.1 Dataset Source

This project uses a real-world energy consumption dataset obtained from **Kaggle**, containing **50,000 smart-meter readings** across **6 key columns**. The dataset is derived from actual household smart-meter measurements and reflects realistic electricity usage patterns over time. Each entry captures a snapshot of consumption and related parameters at a specific timestamp, similar to the format used by energy utilities.

Although the data is historical rather than real-time, it effectively mirrors smart-grid behaviour—showing authentic trends such as:

- Daily usage cycles (morning and evening peaks)
- Weekday vs. weekend differences
- Seasonal temperature-driven variations

Because of this realism, the dataset is well-suited for **time-series analysis**, **load forecasting**, and **machine-learning modelling**, providing a reliable foundation without requiring direct access to live smart-meter infrastructure.

3.2 Data Format

Each data point in the dataset appears in CSV format:

```
1 timestamp,energy_kwh,voltage,current,temperature,humidity,hour,day_of_week,day_of_month,month
2 2024-01-01 06:00:00,227.27808637737408,36.711258841451475,8.343664657989546,3.012377206646746
3 2024-01-01 06:15:00,230.55461294854933,35.98272053961033,8.295982206845673,1.765306630742215,
4 2024-01-01 06:30:00,224.24503211288848,34.711359881823896,7.783850011381628,3.008698028157707
5 2024-01-01 06:45:00,231.87849009172837,10.751360008245166,2.4930091251444813,0.25007926121025
6 2024-01-01 07:00:00,226.99680655040598,5.183725144637466,1.1766890538677461,0.360950699705506
7 2024-01-01 07:15:00,228.5415312510336,24.8293543132293,5.674538654719879,2.26242505460011,0.9
8 2024-01-01 07:30:00,226.00146693885302,24.99459821803897,5.673560515059908,1.748365177829456,
9 2024-01-01 07:45:00,239.092254468,41.531245302562326,9.93682351783646,3.099921581687320,
```

Field Descriptions

COLUMN NAME	DESCRIPTION
timestamp	Date and time of the smart-meter reading
energy_Kwh	Energy consumed during the interval (kWh)
voltage	Measured voltage at the meter (V)
current	Current drawn at the time of reading (A)
temperature	Ambient temperature (°C)
humidity	Relative humidity (%)

This structured format makes the dataset easy to load into machine-learning pipelines and big-data processing tools like Apache Spark

3.3 Data Characteristics

a) Time-Series Nature

The dataset is time-dependent, meaning consumption varies based on:

- Time of day (morning/evening peaks)
- Day of the week
- Temperature and weather
- Human behaviour

This makes the data non-linear and seasonal.

b) Noise & Fluctuations

As with real-world measurements, the dataset may contain:

- Sudden spikes (device turn-on events)
- Drops (energy-saving periods)
- Missing values
- Sensor noise

3.4 How the Dataset is used in this project

Training Phase

- Historical consumption data is cleaned and scaled
- Converted into sequences (e.g., 60 readings → 1 prediction sample)
- These sequences are fed into an LSTM deep-learning model
- The model learns consumption patterns and long-term dependencies.

Streaming Phase

- New data arrives in real time
- Spark processes and aggregates it
- The trained LSTM model receives this new data as input.
- The model predicts the future consumption value

4. System Architecture

The system architecture describes how all parts of the project work together to collect energy data, process it in real time, predict future consumption, and display the results. This project uses a combination of data streaming, big data processing, and deep learning to create an end-to-end smart-grid forecasting system.

4.1 System Workflow

Smart Meter Dataset (Kaggle)



Data Ingestion (Spark Streaming)



Pre-processing & Feature Engineering



Forecasting Models (LSTM & ARIMA)



Visualization Dashboard

Explanation of the flow

- Data is generated continuously
- The data flows through a 'pipe' into Spark
- Spark cleans and organises the data
- The cleaned data is fed into the prediction model
- The model forecasts future energy usage
- The dashboard displays everything in real time.

4.2 Major Components of the Architecture

The system is made up of five main components, each responsible for a specific task. Below, each module is broken down.

4.2.1 Data Source Layer -The system uses historical smart-meter data obtained from Kaggle. The dataset contains time-stamped household energy consumption records structured to resemble real smart-meter readings.

4.2.2 Data Ingestion & Streaming Layer -Apache Spark Streaming processes the historical data in micro-batches to simulate real-time energy data flow. This design prepares the system for future integration with live smart-meter streams.

Responsibilities include:

- Data ingestion
- Time-window batching
- Handling missing values
- Initial data transformation

4.2.3 *Pre-processing & Feature Handling* -The streamed data is transformed into a supervised learning format using a sliding window approach.

- 24-time steps per input sequence
- 10 features per time step.

This layer produces structured input suitable for time-series forecasting models.

Train-Test Split:

After structuring the data, it is divided into training and testing sets to evaluate model performance. Typically, 80% of the data is used for training the model, while the remaining 20% is reserved for testing. This ensures the model learns patterns from historical data and can generalise to unseen energy consumption sequences.

4.2.4 *Forecasting Layer* -Two models are used for energy prediction:

LSTM Model

- Captures temporal dependencies in sequential data
- Handles nonlinear consumption patterns
- Suitable for short-term forecasting

ARIMA Model

- Statistical baseline model
- Captures trends and seasonality
- Used for comparison with deep learning results

Ensemble Models (Random Forest & Gradient Boosted Trees)

In addition to LSTM and ARIMA models, this project incorporates ensemble machine learning models to enhance energy consumption prediction. The ensemble models include:

- Random Forest Regressor (RF)
- Gradient Boosted Trees Regressor (GBT)

These models complement the deep learning (LSTM) and statistical (ARIMA) models by capturing complex interactions between multiple features in the dataset. Together, they form a **hybrid forecasting system** that leverages both temporal patterns and structured feature relationships.

Workflow:

- Relevant features are combined into a single vector using Vector Assembler.
- Random Forest Regressor and Gradient Boosted Trees Regressor are trained on the historical dataset.
- Models are evaluated using RMSE, MAE, and R^2 .
- Trained models are saved for integration into the hybrid predictor, which combines outputs from LSTM and ensemble models for improved forecasting accuracy.

4.2.5 *Dashboard / Visualisation* -The system includes a **visualisation component** to assess model performance using plots generated by the visualise results function. This allows a visual inspection of prediction accuracy and model behaviour.

Visualization Methods

- Time Series plot
 - i) Compares actual energy consumption with predicted values over time.
 - ii) Helps identify periods where the model performs well or struggles.
- Scatter plot
 - i) Plots predicted values against actual consumption values.
 - ii) Shows correlation, dispersion, and potential bias in predictions.

5 Model Evaluation and Results in Energy Consumption Forecasting for Smart Grids

Model evaluation in smart grid energy forecasting is all about checking if your prediction system is actually smart or just pretending. Since power demand can swing wildly due to weather, human behaviour and random chaos, evaluation must be rigorous, multi-metric and sensitive to both short term spikes and long-term patterns.

5.1 Training Data Preparation

The LSTM model was trained using time-series sequences generated from historical smart-meter data. A sliding window approach was applied to capture temporal dependencies in energy consumption.

- Training samples: 29503
- Time steps per sample: 24
- Features per time step: 10

Final data shapes:

- X_train: (29503, 24, 10)
- Y_train: (29503)

This structure allows the LSTM network to learn consumption patterns over a 24-time-step horizon using multiple influencing features.

5.2 Evaluation Metrics

The following regression metrics were used to evaluate model performance:

1. *Mean Absolute Error (MAE)*: Measures the average absolute difference between predicted and actual consumption. Lower MAE indicates stability, even when data fluctuates.
2. *Mean Squared Error (MSE)*: Measures the average squared difference between predicted and actual energy values. Lower values indicate better accuracy.
3. *Loss(Training Objective)*: The loss function optimized during training, closely related to MSE.

5.3 Training Results

The LSTM model was trained for up to **50 epochs**, with early stopping behaviour observed as validation performance stabilised.

Best Observed Validation Performance

- Validation Loss: ≈ 0.434
- Validation MAE: ≈ 0.813
- Validation MSE: ≈ 1.000

These results indicate that the model achieved consistent prediction accuracy on unseen validation data.

5.4 Training vs Validation Analysis

- Training and validation losses remained **close throughout training**, indicating **minimal overfitting**.
- Performance improvements plateaued after approximately **8–11 epochs**, suggesting the model reached convergence early.
- Learning rate reduction improved stability without causing degradation in validation accuracy.

This behaviour confirms that the LSTM successfully learned temporal patterns without memorising the training data.

5.5 Model Stability and Generalisation

- The small gap between training and validation MAE shows good generalisation.
- Stable validation metrics across epochs indicate robustness to time-series fluctuations.
- The model performs well for short-term energy consumption forecasting in a smart-grid environment.

6. Installation and Usage

6.1 Environment Setup and System Execution

To set up and run the forecasting system, follow these general steps:

1. *Environment Setup*: Ensure you have VS Code (or any preferred IDE), Python 3.14 (or any version of Python), Apache Spark, NodeJS and a JDK installed.

2. *Dependencies:* To run the model, you'll need to have some dependencies installed. Install the required Python package using the requirements.txt file by running this in your terminal:
`pip install -r requirements.txt`

The key dependencies include TensorFlow, PyTorch, Flask, PySpark, Numpy and Matplotlib

3. *System Start:* Run the main script “python main.py”, which initialises the Spark environment, starts the data streaming (or simulation), and deploys the forecasting model.
4. *Monitoring:* Access the local web dashboard to visualise real-time energy predictions and assess model performance.

6.2 Tools and Framework Usage

The following tools were installed to support data processing, modelling, and the project:

- *TensorFlow* – used to build and train the LSTM model for energy consumption forecasting.
- *PyTorch* – used for experimentation and comparison of deep learning models.
- *PySpark* – used for large-scale data processing and streaming within the smart grid framework.
- *NumPy* – used for numerical computations and data pre-processing.
- *Matplotlib* – used to visualise energy consumption trends and model results.

These tools together enabled efficient data handling, model training, and result analysis for smart grid energy forecasting.

6 Conclusion

This project shows how real-time data streaming and machine-learning models can work together to predict energy consumption more accurately. Using Spark Streaming to process fast incoming data and models like LSTM and ARIMA to make forecasts, the system gives a clearer picture of energy usage as it changes. Extra features like weather and time-of-day make the predictions feel more realistic, and the dashboard helps simply visualise everything.

However, the system has several limitations. One major challenge was obtaining appropriate training data, and the available feature set is somewhat limited. As a result, certain human behaviours and rare events are not fully captured by the models. Additionally, the LSTM model can become computationally slow when scaled to larger datasets, while ARIMA struggles with noisy or highly unpredictable patterns. External factors such as extreme weather conditions or sudden load changes may further reduce prediction accuracy in real-world deployments. Finally, the visualisation dashboard would require more robust infrastructure to support large-scale, city-wide usage.

In summary, the project performs effectively as a prototype, demonstrating the feasibility of smart-grid energy forecasting. However, there is significant room for improvement to make the system faster, more intelligent, and more adaptable for real-world energy grid applications.